

PROGRAMOWANIE OBIEKTOWE

Projekt

Wydział Informatyki i Telekomunikacji	Kierunek: Informatyka Techniczna
Grupa zajęciowa (np. Pn7:30):Czw 13:15	Semestr: 2025/26 LATO
Nazwisko i Imię:Filip Klich	Nr indeksu:293446
Nazwisko i Imię:Michał Sikora	Nr indeksu:292845
Nazwisko i Imię:	Nr indeksu:
Nr. grupy projektowej:3	
Prowadzący: mgr inż. Karol Puchała	

TEMAT:

„Objectively Worse Doom”

OCENA:

PUNKTY:

Data:

1

2

3

backend.h

```
#pragma once
#include <vector>
#include <bitset>
#include <linux/input.h>
#include "input/input-backend.h"

namespace input {
    namespace evdev {
        class backend : public input::input_backend {
        private:
            struct device_node {
                int fd;
                bool is_keyboard;
                bool is_mouse;
            };

            std::vector<device_node> devices;
            std::bitset<KEY_CNT> key_map;
            mouse_state mouse;

            int last_abs_x;
            int last_abs_y;

            int epoll_fd;

            uint16_t map_to_native(key k) const;
            void probe_devices();
            void probe_device(std::string const& dev_name);
            void process_event(struct input_event const& ev);

        public:
            backend();
            ~backend() override;

            void update() override;
            bool is_key_down(key k) const override;
            mouse_state get_mouse_state() const override;
            void reset_mouse_state(int nx, int ny) override;
        };
    }
}
```


input-backend.h

```
#pragma once
```

```
#include <stdint>
```

```
namespace input {  
    enum class key {  
        unknown = 0,  
        a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z,  
        n0, n1, n2, n3, n4, n5, n6, n7, n8, n9,  
        f1, f2, f3, f4, f5, f6, f7, f8, f9, f10, f11, f12,  
        esc, tab, caps_lock, l_shift, l_ctrl, l_meta, l_alt, space, r_alt, r_meta, r_ctrl, r_shift,  
        print_screen, scroll_lock, pause, insert, home, page_up, del, end, page_down, up, down, left,  
        backtick, hyphen, equals, open_bracket, close_bracket, backslash, semicolon, quote, comma, p  
        num_lock, kp_slash, kp_asterisk, kp_minus, kp_plus, kp_enter, kp_dot, kp0, kp1, kp2, kp3, kp  
    };  
  
    struct mouse_state {  
        int x, y;  
        bool left, right, middle;  
    };  
  
    class input_backend {  
    protected:  
        bool bad;  
    public:  
        input_backend();  
        virtual ~input_backend() = default;  
  
        virtual void update() = 0;  
        virtual bool is_key_down(key k) const = 0;  
        virtual mouse_state get_mouse_state() const = 0;  
        virtual void reset_mouse_state(int nx = 0, int ny = 0) = 0;  
        virtual bool is_bad() const;  
  
        static key to_key(char c);  
    };  
}
```

crtc.h

```
#pragma once

#include <stdint>
#include "device-context.h"
#include "mode.h"

namespace rendering {
    namespace drm_kms {
        class crtc {
            device_context const& dev;
            std::uint32_t crtc_id;

        public:
            crtc(device_context const& d, uint32_t id);

            bool set_config(uint32_t fb_id, uint32_t conn_id, mode const& m);

            bool page_flip(uint32_t fb_id) const;
        };
    }
}
```

framebuffer.h

```
#pragma once

#include <stdint>
#include <stddef>
#include "device-context.h"
#include "mode.h"

namespace rendering {
    namespace drm_kms {
        class crtc;
        class connector;
        class backend;

        class framebuffer {
            device_context const& dev;
            uint32_t handle;

            uint32_t fb_id;
            uint32_t* mmio_ptr;
            size_t size;
            uint32_t pitch;

        public:
            framebuffer(device_context const& d, uint32_t width, uint32_t height, bool *const success) : dev(d), handle(0), fb_id(0), mmio_ptr(0), size(0), pitch(0) {}
            ~framebuffer();

            void copy_from(uint32_t const* src, size_t pixel_count) const;

            bool flip_onto(crtc const& c) const;

            bool apply_config(crtc &c, std::uint32_t const connector_id, mode const& m) const;

            friend connector;
            friend backend;
        };
    }
}
```

mode.h

```
#pragma once

#include <drm/drm.h>
#include "rendering/rendering-mode.h"

namespace rendering {
    namespace drm_kms {
        class mode : public rendering::rendering_mode {
            struct drm_mode_modeinfo kernel_mode;

        public:

            mode(struct drm_mode_modeinfo const& info);
            ~mode() override = default;

            struct drm_mode_crtc create_crtc_req(std::uint32_t const crtc_id, std::uint32_t const fb_id) const;

            unsigned int operator()(util::component_tag<"x_res">) const override;
            unsigned int operator()(util::component_tag<"y_res">) const override;
            unsigned int operator()(util::component_tag<"refresh_hz">) const override;
            bool operator()(util::component_tag<"has_vsync">) const override;
        };
    }
}
```

connector.h

```
#pragma once

#include <vector>
#include <memory>
#include <cstdint>
#include "device-context.h"
#include "mode.h"

namespace rendering {
    namespace drm_kms {
        class crtc;
        class framebuffer;

        class connector {
            device_context const& dev;
            uint32_t connector_id;
            uint32_t encoder_id;

        public:
            connector(device_context const& d, uint32_t id, bool* const success = nullptr);

            std::vector<std::unique_ptr<mode const>> probe_modes();

            bool apply_config(crtc& c, framebuffer const& fb, mode const& m) const;
        };
    }
}
```

device-context.h

```
#pragma once

#include <string>

namespace rendering {
    namespace drm_kms {
        class device_context {
            int fd;

        public:

            explicit device_context(std::string const& path);
            ~device_context();

            bool is_valid() const;
            int ioctl(unsigned long request, void* arg) const;
            void *mmap(void *addr, size_t len, int prot, int flags, off_t off) const;
        };
    }
}
```

backend.h

```
#pragma once

#include <memory>
#include <vector>
#include <drm/drm.h>

#include "rendering/rendering-backend.h"
#include "device-context.h"
#include "framebuffer.h"
#include "connector.h"
#include "crtc.h"

namespace rendering {
    namespace drm_kms {
        class backend : public rendering::rendering_backend {
        protected:
            std::unique_ptr<device_context> dev;
            std::unique_ptr<connector> active_connector;
            std::unique_ptr<crtc> active_crtc;
            bool is_bad;

            std::unique_ptr<framebuffer> buffers[2];
            int front_buffer_index;
            std::vector<uint32_t> shadow;

            std::unique_ptr<mode const> current_mode;

            uint32_t original_fb_id = 0;
            uint32_t original_connector_id = 0;
            struct drm_mode_modeinfo original_mode = {};
            bool has_original_state = false;

        public:
            backend();
            ~backend() override;

            bool bad() const override;
            virtual std::vector<std::unique_ptr<rendering_mode const>> operator()(util::component_tag<"m
            void push_mode(std::unique_ptr<rendering_mode const> pushed_mode) override;
            unsigned int operator()(util::component_tag<"width">) override;
            unsigned int operator()(util::component_tag<"height">) override;
            unsigned int operator()(util::component_tag<"pitch">) override;
            std::uint32_t *operator()(util::component_tag<"mmio">) override;
            void wait_for_vsync() override;
            void flush() override;
        };
    }
}
```

renderer-2d.h

```
#pragma once
```

```
#include "rendering-backend.h"
#include "assets/asset-manager.h"
#include "assets/texture.h"
#include <string_view>
#include <cstdint>
#include <algorithm>
```

```
namespace rendering {
    class renderer_2d {
        rendering_backend &target;
        assets::asset_manager const& tex_manager;
        assets::texture const& font_texture;

    public:
        renderer_2d(rendering_backend &tgt, assets::asset_manager const& tm, assets::texture const& font_texture) : target(tgt), tex_manager(tm), font_texture(font_texture) {}

        void draw_texture(assets::texture const& tex, int x, int y, int w, int h) const;

        void draw_text(std::string_view text, int x, int y, int char_w, int char_h, std::uint32_t color) const;

        void draw_rect(int x, int y, int w, int h, std::uint32_t color) const;
    };
}
```


lighting.h

```
#pragma once
```

```
#include <cstdint>
```

```
namespace rendering {
```

```
    class lighting {
```

```
        static constexpr float dl_start    = 64.0f;
```

```
        static constexpr float dl_density = 0.25f;
```

```
    public:
```

```
        __attribute__((always_inline)) static inline int calculate(std::uint8_t const sector_light_l
```

```
        if(depth > dl_start) {
```

```
            int corrected_light = sector_light_level - (depth - dl_start) * dl_density;
```

```
            if(corrected_light < 0) corrected_light = 0;
```

```
            return corrected_light;
```

```
        } else {
```

```
            return sector_light_level;
```

```
        }
```

```
    }
```

```
        __attribute__((always_inline)) static inline std::uint32_t apply(std::uint32_t const orig, i
```

```
        std::uint32_t swar_rb = orig & 0x00ff00ff;
```

```
        std::uint32_t swar_g  = orig & 0x0000ff00;
```

```
        swar_rb = ((swar_rb * light) >> 8) & 0x00ff00ff;
```

```
        swar_g  = ((swar_g  * light) >> 8) & 0x0000ff00;
```

```
        return swar_rb | swar_g;
```

```
    }
```

```
};
```

```
}
```

frd.h

```
#pragma once

#include <cstdint>
#include <vector>
#include "math/vec2.h"

namespace rendering {
    // All per-frame, camera-derived constants that rendering objects need.
    // Computed once in software_renderer::render_bsp() and passed down by value.
    struct frame_rendering_data {
        math::vec2 cam_pos;
        float cam_height, cam_angle;
        unsigned int sw, sh;
        float half_sw;
        unsigned int pitch;
        std::uint32_t *__restrict mmio;
        float fov_scale;
        float inv_fov_scale;
        float cos_cam_angle, sin_cam_angle;

        // Per-column Euclidean distance correction factor ( $\sqrt{dx^2 + 1}$ ).
        // Stored here so visplanes and vissprites can access it without going
        // back through software_renderer.
        std::vector<float> const* euclidian_dist_factor;
    };
}...
```

sprite.h

```
#pragma once

#include "math/vec2.h"
#include "assets/ids.h"
#include "util/componentized.h"

namespace rendering {
    class software_renderer;
    class vissprite;

    class sprite : public util::componentized<sprite> {
    protected:

        [[=util::ref_component_field{}]] math::vec2 pos;
        [[=util::ref_component_field{}]] float z_pos;
        [[=util::ref_component_field{}]] float angle;
        assets::texture_id tex_id;
        float inherent_scale;
        float hit_flash = 0.0f;

    public:

        sprite(math::vec2 const p, float const z, assets::texture_id const tex, float const is);
        //sprite();
        virtual ~sprite() = default;

        friend util::componentized<sprite>;
        friend software_renderer;
        friend vissprite;
    };
}
```

rendering-backend.h

```
#pragma once

#include <vector>
#include <memory>
#include <cstddef>
#include <cstdint>

#include "util/componentized.h"
#include "rendering-mode.h"

namespace rendering {
    class rendering_backend {
    public:

        virtual bool bad() const = 0;
        virtual std::vector<std::unique_ptr<rendering_mode const>> operator()(util::component_tag<"m
        virtual void push_mode(std::unique_ptr<rendering_mode const> mode) = 0;
        virtual unsigned int operator()(util::component_tag<"width">) = 0;
        virtual unsigned int operator()(util::component_tag<"height">) = 0;
        virtual unsigned int operator()(util::component_tag<"pitch">) = 0;
        virtual std::uint32_t *operator()(util::component_tag<"mmio">) = 0;
        virtual void wait_for_vsync() = 0;
        virtual void flush() = 0;

        virtual ~rendering_backend() = default;
    };
}
```

rendering-mode.h

```
#pragma once

#include "util/componentized.h"

namespace rendering {
    class rendering_mode : public util::componentized<rendering_mode> {
    public:
        virtual ~rendering_mode() = default;

        virtual unsigned int operator()(util::component_tag<"x_res">) const = 0;
        virtual unsigned int operator()(util::component_tag<"y_res">) const = 0;
        virtual unsigned int operator()(util::component_tag<"refresh_hz">) const = 0;
        virtual bool operator()(util::component_tag<"has_vsync">) const = 0;

        friend util::componentized<rendering_mode>;
    };
}
```

software-renderer.h

```
#pragma once

#include "rendering-backend.h"
#include "frd.h"
#include "visplane.h"
#include "vissprite.h"
#include "assets/asset-manager.h"
#include "geometry/linedef.h"
#include "geometry/bsp-node.h"
#include "geometry/map-data.h"
#include "math/vec2.h"
#include <vector>
#include <cstdint>

namespace rendering {
    class software_renderer {
        rendering_backend &target;
        assets::asset_manager const& tex_manager;
        geometry::map_data const& current_map;

        static constexpr float near_z = 0.1f;

        std::vector<int> upper_clip;
        std::vector<int> lower_clip;
        std::vector<visplane> visplanes;
        std::vector<vissprite> vissprites;

        std::vector<float> euclidian_dist_factor;

        void render_bsp_node(util::indexed_storage<geometry::bsp_node>::id_t node_id, frame_rendering_data frd);
        void project_and_draw_linedef(geometry::linedef line, frame_rendering_data const& frd);
        void draw_solid_wall_span(float proj_x1, float proj_x2, float z1, float z2, float u1, float u2, frame_rendering_data frd);
        void draw_portal_wall_span(float proj_x1, float proj_x2, float z1, float z2, float u1, float u2, frame_rendering_data frd);
        void render_visplanes(frame_rendering_data const& frd);
        void add_vissprite(sprite *const s, std::uint8_t light, frame_rendering_data const& frd);
        void render_vissprites(frame_rendering_data const& frd);
        bool is_box_visible(geometry::bsp_node::bounding_box const& box, frame_rendering_data const& frd);

    public:
        software_renderer(rendering_backend &tgt, assets::asset_manager const& tm, geometry::map_data const& md);

        void render_bsp(math::vec2 const cam_pos, float const cam_height, float cam_angle, float fov);
    };
}
```

vissprite.h

```
#pragma once

#include <cstdint>
#include <vector>

#include "assets/asset-manager.h"
#include "sprite.h"

namespace rendering {
    struct frame_rendering_data;

    class vissprite {
        float depth;
        int cx1, cx2;
        float proj_x, scale;
        float z_pos;
        assets::texture_id tex_id;
        std::uint8_t light_level;
        bool flash_red;

        std::vector<int> upper_clip;
        std::vector<int> lower_clip;

    public:

        vissprite(sprite const& sprite, float const z, int const clamped_x1, int const clamped_x2, f

        static void pa_sort(std::vector<vissprite> &vec);

        void render(assets::asset_manager const& tex_manager, frame_rendering_data const& frd) const
    };
}
```

visplane.h

```
#pragma once
```

```
#include <cstdint>
```

```
#include <vector>
```

```
#include "assets/asset-manager.h"
```

```
#include "assets/ids.h"
```

```
namespace rendering {
```

```
    struct frame_rendering_data;
```

```
    class visplane {
```

```
        float height;
```

```
        assets::texture_id tex_id;
```

```
        std::uint8_t light_level;
```

```
        int min_x, max_x;
```

```
        std::vector<int> top, bottom;
```

```
    public:
```

```
        visplane(unsigned int const sw, float const h, assets::texture_id const tid, std::uint8_t co
```

```
        static void add_column(std::vector<visplane>& pool, int x, int y_start, int y_end, unsigned
```

```
        void render(assets::asset_manager const& tex_manager, frame_rendering_data const& frd) const
```

```
    };
```

```
}
```


ray2.h

```
#pragma once

#include "vec2.h"
#include "segment.h"

namespace math {
    class ray2 {
    public:
        vec2 end;
        vec2 direction;

        constexpr ray2(vec2 const e, vec2 const d);

        __attribute__((always_inline)) constexpr inline bool operator==(ray2 const other) const;
        __attribute__((always_inline)) constexpr inline bool operator!=(ray2 const other) const;

        static ray2 const rotate(ray2 const ray, float const angle);

        bool intersects(segment const seg, vec2 &hit_point, float &distance, float &seg_len) const;
    };
}

constexpr math::ray2::ray2(vec2 const e, vec2 const d) : end(e), direction(d) {}

__attribute__((always_inline)) constexpr inline bool
math::ray2::operator==(ray2 const other) const {
    return end == other.end && direction == other.direction;
}

__attribute__((always_inline)) constexpr inline bool
math::ray2::operator!=(ray2 const other) const {
    return !(*this == other);
}
```

segment.h

```
#pragma once

#include "vec2.h"
#include "util/componentized.h"

namespace math {
    class segment : public util::componentized<segment> {
        [[=util::component_field{}}]] vec2 point0, point1;

    public:

        constexpr segment(vec2 const p0, vec2 const p1);

        __attribute__((always_inline)) constexpr inline bool operator==(segment const other) const;
        __attribute__((always_inline)) constexpr inline bool operator!=(segment const other) const;

        float angle() const;
        constexpr vec2 const midpoint() const;
        float len() const;
        float sqr_len() const;

        friend util::componentized<segment>;
    };
}

constexpr math::segment::segment(vec2 const p0, vec2 const p1) : point0(p0), point1(p1) {}

constexpr math::vec2 const math::segment::midpoint() const {
    return (point0 + point1) / 2.0f;
}

__attribute__((always_inline)) constexpr inline bool
math::segment::operator==(segment const other) const {
    return point0 == other.point0 && point1 == other.point1;
}

__attribute__((always_inline)) constexpr inline bool
math::segment::operator!=(segment const other) const {
    return !(*this == other);
}
```

vec3.h

```
#pragma once
```

```
#include "util/componentized.h"
```

```
#include "vec2.h"
```

```
namespace math {
```

```
    class vec3 : public util::componentized<vec3> {
```

```
        [[=util::ref_component_field{}]] float x;
```

```
        [[=util::ref_component_field{}]] float y;
```

```
        [[=util::ref_component_field{}]] float z;
```

```
    public:
```

```
        constexpr inline vec3(float const X = 0, float const Y = 0, float const Z = 0);
```

```
        constexpr inline vec3(vec2 const flat, float const Z = 0);
```

```
        __attribute__((always_inline)) constexpr inline vec3 const operator+(vec3 const other) const;
```

```
        __attribute__((always_inline)) constexpr inline vec3 const operator-(vec3 const other) const;
```

```
        __attribute__((always_inline)) constexpr inline vec3 const operator-() const;
```

```
        __attribute__((always_inline)) constexpr inline vec3 const operator*(float const scalar) const;
```

```
        __attribute__((always_inline)) constexpr inline vec3 const operator/(float const scalar) const;
```

```
        __attribute__((always_inline)) constexpr inline vec3 &operator+=(vec3 const other);
```

```
        __attribute__((always_inline)) constexpr inline vec3 &operator-=(vec3 const other);
```

```
        __attribute__((always_inline)) constexpr inline vec3 &operator*=(float const scalar);
```

```
        __attribute__((always_inline)) constexpr inline vec3 &operator/=(float const scalar);
```

```
        __attribute__((always_inline)) constexpr inline bool operator==(vec3 const other) const;
```

```
        __attribute__((always_inline)) constexpr inline bool operator!=(vec3 const other) const;
```

```
        static constexpr float dot_product(vec3 const first, vec3 const second);
```

```
        static constexpr vec3 const cross_product(vec3 const first, vec3 const second);
```

```
        constexpr float len() const;
```

```
        constexpr float sqr_len() const;
```

```
        constexpr vec3 const normalized() const;
```

```
        friend util::componentized<vec3>;
```

```
    };
```

```
}
```

```
constexpr inline
```

```
math::vec3::vec3(float const X, float const Y, float const Z) : x(X), y(Y), z(Z) {}
```

```
constexpr inline
```

```
math::vec3::vec3(vec2 const flat, float const Z) : x(flat("x"_f)), y(flat("y"_f)), z(Z) {}
```

```
__attribute__((always_inline)) constexpr inline math::vec3 const
```

```
math::vec3::operator+(vec3 const other) const {
```

```
    return vec3(x + other.x, y + other.y, z + other.z);
```

```
}
```

```
__attribute__((always_inline)) constexpr inline math::vec3 const
```

```
math::vec3::operator-(vec3 const other) const {
```

```
    return vec3(x - other.x, y - other.y, z - other.z);
```

```
}
```

```

__attribute__((always_inline)) constexpr inline math::vec3 const
math::vec3::operator-() const {
    return vec3(-x, -y, -z);
}

__attribute__((always_inline)) constexpr inline math::vec3 const
math::vec3::operator*(float const scalar) const {
    return vec3(x * scalar, y * scalar, z * scalar);
}

__attribute__((always_inline)) constexpr inline math::vec3 const
math::vec3::operator/(float const scalar) const {
    return vec3(x / scalar, y / scalar, z / scalar);
}

__attribute__((always_inline)) constexpr inline math::vec3 &
math::vec3::operator+=(vec3 const other) {
    x += other.x; y += other.y; z += other.z;
    return *this;
}

__attribute__((always_inline)) constexpr inline math::vec3 &
math::vec3::operator-=(vec3 const other) {
    x -= other.x; y -= other.y; z -= other.z;
    return *this;
}

__attribute__((always_inline)) constexpr inline math::vec3 &
math::vec3::operator*=(float const scalar) {
    x *= scalar; y *= scalar; z *= scalar;
    return *this;
}

__attribute__((always_inline)) constexpr inline math::vec3 &
math::vec3::operator/=(float const scalar) {
    x /= scalar; y /= scalar; z /= scalar;
    return *this;
}

__attribute__((always_inline)) constexpr inline bool
math::vec3::operator==(vec3 const other) const {
    return x == other.x && y == other.y && z == other.z;
}

__attribute__((always_inline)) constexpr inline bool
math::vec3::operator!=(vec3 const other) const {
    return !(*this == other);
}

constexpr float math::vec3::dot_product(vec3 const first, vec3 const second) {
    return first.x * second.x + first.y * second.y + first.z * second.z;
}

constexpr math::vec3 const math::vec3::cross_product(vec3 const first, vec3 const second) {
    return vec3(
        first.y * second.z - first.z * second.y,
        first.z * second.x - first.x * second.z,

```

```
        first.x * second.y - first.y * second.x  
    );  
}
```

vec2.h

```
#pragma once
#include "util/componentized.h"

namespace math {
    class vec3;

    class vec2 : public util::componentized<vec2> {
        [[=util::ref_component_field{}]] float x;
        [[=util::ref_component_field{}]] float y;

    public:

        constexpr inline vec2(float const X = 0, float const Y = 0);
        vec2(vec3 const& vec);

        __attribute__((always_inline)) constexpr inline vec2 operator+(vec2 const other) const;
        __attribute__((always_inline)) constexpr inline vec2 operator-(vec2 const other) const;
        __attribute__((always_inline)) constexpr inline vec2 operator-() const;
        __attribute__((always_inline)) constexpr inline vec2 operator*(float const scalar) const;
        __attribute__((always_inline)) constexpr inline vec2 operator/(float const scalar) const;

        __attribute__((always_inline)) constexpr inline vec2 &operator+=(vec2 const other);
        __attribute__((always_inline)) constexpr inline vec2 &operator-=(vec2 const other);
        __attribute__((always_inline)) constexpr inline vec2 &operator*=(float const scalar);
        __attribute__((always_inline)) constexpr inline vec2 &operator/=(float const scalar);

        __attribute__((always_inline)) constexpr inline bool operator==(vec2 const other) const;
        __attribute__((always_inline)) constexpr inline bool operator!=(vec2 const other) const;

        static float dot_product(vec2 const first, vec2 const second);
        static float cross_product(vec2 const first, vec2 const second);

        float len() const;
        float sqr_len() const;
        vec2 const normalized() const;
        vec2 const perpendicular() const;
        float angle() const;

        static float angle_between(vec2 const first, vec2 const second);
        static vec2 const rotate(vec2 const vec, float const angle);
        static vec2 const rotate_with_known_trig(vec2 const vec, float const cos, float const sin);

        friend util::componentized<vec2>;
    };
}

constexpr inline
math::vec2::vec2(float const X, float const Y) : x(X), y(Y) {}

__attribute__((always_inline)) constexpr inline math::vec2
math::vec2::operator+(vec2 const other) const {
    return vec2(x + other.x, y + other.y);
}

__attribute__((always_inline)) constexpr inline math::vec2
```

```

math::vec2::operator-(vec2 const other) const {
    return vec2(x - other.x, y - other.y);
}

__attribute__((always_inline)) constexpr inline math::vec2
math::vec2::operator-() const {
    return vec2(-x, -y);
}

__attribute__((always_inline)) constexpr inline math::vec2
math::vec2::operator*(float const scalar) const {
    return vec2(x * scalar, y * scalar);
}

__attribute__((always_inline)) constexpr inline math::vec2
math::vec2::operator/(float const scalar) const {
    return vec2(x / scalar, y / scalar);
}

__attribute__((always_inline)) constexpr inline math::vec2&
math::vec2::operator+=(vec2 const other) {
    x += other.x;
    y += other.y;
    return *this;
}

__attribute__((always_inline)) constexpr inline math::vec2&
math::vec2::operator-=(vec2 const other) {
    x -= other.x;
    y -= other.y;
    return *this;
}

__attribute__((always_inline)) constexpr inline math::vec2&
math::vec2::operator*=(float const scalar) {
    x *= scalar;
    y *= scalar;
    return *this;
}

__attribute__((always_inline)) constexpr inline math::vec2&
math::vec2::operator/=(float const scalar) {
    x /= scalar;
    y /= scalar;
    return *this;
}

__attribute__((always_inline)) constexpr inline bool
math::vec2::operator==(vec2 const other) const {
    return x == other.x && y == other.y;
}

__attribute__((always_inline)) constexpr inline bool
math::vec2::operator!=(vec2 const other) const {
    return !(*this == other);
}

```

file-resource.h

```
#pragma once
#define FILE_RESOURCE_H

#include <vector>
#include "resource.h"

namespace util {
    class file_resource : public resource {
        std::vector<std::byte> file_data;

    public:
        explicit file_resource(std::vector<std::byte> data);
        ~file_resource() override = default;

        void const* operator()(component_tag<"beginning">) const override;
        void const* operator()(component_tag<"end">) const override;
        std::size_t operator()(component_tag<"size">) const override;
    };
}
```


resource.h

```
#pragma once
#define RESOURCE_H

#include <cstddef>
#include "util/componentized.h"

namespace util {
    class resource {
    public:
        virtual ~resource() = default;

        virtual void const* operator()(component_tag<"beginning">) const = 0;
        virtual void const* operator()(component_tag<"end">) const = 0;
        virtual std::size_t operator()(component_tag<"size">) const = 0;
    };
}
```

binary-resource.h

```
#pragma once
#define BINARY_RESOURCE_H

#include "resource.h"

namespace util {
    class binary_resource : public resource {
        void const* begin_ptr;
        void const* end_ptr;
        std::size_t byte_size;

    public:

        binary_resource(void const* begin, void const* end);
        ~binary_resource() override = default;

        void const* operator()(component_tag<"beginning">) const override;
        void const* operator()(component_tag<"end">) const override;
        std::size_t operator()(component_tag<"size">) const override;
    };
}
```

componentized.h

```
#pragma once
#define COMPONENTIZED_H

#include <concepts>
#include <meta>
#include <iostream>
#include <string_view>
#include <vector>

namespace util {
    struct component_field {};
    struct ref_component_field {};

    template<std::size_t N>
    struct fixed_string {
        char data[N];
        consteval fixed_string(char const (&s)[N]) {
            for (std::size_t i = 0; i < N; ++i) data[i] = s[i];
        }
        consteval std::string_view view() const { return {data, N - 1}; }
    };

    template<fixed_string field>
    struct component_tag {};

    template <typename T>
    class componentized {
        template <fixed_string field, typename A>
        consteval static std::meta::info get_tagged_member() {
            auto ctx = std::meta::access_context::current();
            auto members = std::meta::nonstatic_data_members_of(^T, ctx);

            for (auto m : members) {
                if (std::meta::identifier_of(m) == field.view()) {
                    auto attrs = std::meta::annotations_of(m);
                    for (auto attr : attrs) {
                        if (std::meta::type_of(attr) == ^A) {
                            return m;
                        }
                    }
                }
            }
            return {};
        }

    public:
        template<fixed_string field>
        requires (get_tagged_member<field, ref_component_field const>() != std::meta::info{})
        auto& operator()(component_tag<field>) {
            constexpr auto target = get_tagged_member<field, ref_component_field const>();
            return static_cast<T *>(this)->[:target:];
        }

        template<fixed_string field>
        requires
```

```

        ( (get_tagged_member<field, component_field const>()      != std::meta::info{}) ||
          (get_tagged_member<field, ref_component_field const>() != std::meta::info{}) )
    auto const& operator()(component_tag<field>) const {
        constexpr auto target = []{
            auto tgt = get_tagged_member<field, component_field const>();
            if(tgt != std::meta::info{}) return tgt;
            return get_tagged_member<field, ref_component_field const>();
        }();
        return static_cast<T const *>(this)->[:target:];
    }
};

/* usage:
 * class test : public util::componentized<test> {
 *     [[=util::ref_component_field{}]] int hp;
 *     [[=util::component_field{}]] int max_hp;
 *
 *     float secret;
 *
 *     [[=util::component_field{}]] float speed;
 *
 *     friend class componentized<test>;
 * };
 *
 * int main() {
 *     test d(100, 5.5f);
 *
 *     std::cout << "Initial HP: " << d("hp"_f) << std::endl;
 *     d("hp"_f) = 150;
 *     std::cout << "Updated HP: " << d("hp"_f) << std::endl;
 *     std::cout << "Speed: " << d("speed"_f) << std::endl;
 * }
 */

}

/* must be global */
template<util::fixed_string field>
constexpr util::component_tag<field> operator""_f() {
    return {};
}

```

resource-loader.h

```
#pragma once

/* auto-generated file with all declarations */
#include "res.h"

#include <string>
#include <string_view>
#include <unordered_map>
#include <mutex>
#include <shared_mutex>
#include <memory>

#if __has_include(<meta>)
#include <meta>
#endif

#include "resource.h"
#include "binary-resource.h"
#include "file-resource.h"

namespace util {
    class resource_loader {
        std::unordered_map<std::string, void*> symbol_cache;
        std::unordered_map<std::string, std::unique_ptr<resource>> resource_cache;
        mutable std::shared_mutex cache_mutex;

        void* lookup_binary(std::string_view resource_name);
        std::unique_ptr<resource> load_file(std::string_view resource_name) const;
        resource* lookup_resource_runtime(std::string_view resource_name);

        constexpr std::string make_symbol_name(std::string_view resource_name) const;

    #if __has_include(<meta>)
        static constexpr std::size_t count_res_members();

        template <std::size_t N>
        struct info_array {
            std::meta::info data[N > 0 ? N : 1];
        };

        template <std::size_t N>
        static constexpr info_array<N> get_res_members_array();

        constexpr resource const* get_by_symbol_reflection(std::string_view symbol_name) const;
    #endif

    public:
        resource_loader() = default;
        ~resource_loader() = default;

        resource_loader(resource_loader const&) = delete;
        resource_loader& operator=(resource_loader const&) = delete;

        constexpr resource* lookup_resource(std::string_view resource_name);
    };
};
```

```

constexpr std::string resource_loader::make_symbol_name(std::string_view resource_name) const {
    std::string sym = "_binary_res_";
    sym.reserve(sym.size() + resource_name.size());

    for (char ch : resource_name) {
        unsigned char c = static_cast<unsigned char>(ch);
        if ((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || (c >= '0' && c <= '9')) {
            sym += ch;
        } else {
            sym += '_';
        }
    }
    return sym;
}

#if __has_include(<meta>)
constexpr std::size_t resource_loader::count_res_members() {
    std::size_t count = 0;
    for (auto m : std::meta::members_of(^res_symbols, std::meta::access_context::current())) {
        if (std::meta::has_identifier(m)) {
            if (std::string_view(std::meta::identifier_of(m)).starts_with("_binary_res_")) {
                ++count;
            }
        }
    }
    return count;
}

template <std::size_t N>
constexpr resource_loader::info_array<N> resource_loader::get_res_members_array() {
    resource_loader::info_array<N> arr{};
    std::size_t idx = 0;
    for (auto m : std::meta::members_of(^res_symbols, std::meta::access_context::current())) {
        if (std::meta::has_identifier(m)) {
            if (std::string_view(std::meta::identifier_of(m)).starts_with("_binary_res_")) {
                if (idx < N) {
                    arr.data[idx++] = m;
                }
            }
        }
    }
    return arr;
}

constexpr resource const* resource_loader::get_by_symbol_reflection(std::string_view symbol_name)
    if constexpr {
#if __has_include(<meta>)
        static constexpr std::size_t num_members = count_res_members();
        static constexpr auto members_arr = get_res_members_array<num_members>();

        if (!symbol_name.ends_with("_start")) {
            return nullptr;
        }
        std::string end_name(symbol_name.substr(0, symbol_name.size() - 6));
        end_name += "_end";

        unsigned char const* begin_sym = nullptr;

```

```

        unsigned char const* end_sym    = nullptr;

        /* TODO: work around a gcc 16.1.1 20260430 bug */
#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wshadow"
        template for (constexpr auto m : members_arr.data) {
            if constexpr (std::meta::has_identifier(m)) {
                constexpr std::string_view name = std::meta::identifier_of(m);
                if (name == symbol_name) {
                    begin_sym = [: m :];
                } else if (name == end_name) {
                    end_sym = [: m :];
                }
            }
        }
#pragma GCC diagnostic pop

        if (begin_sym && end_sym) {
            return std::define_static_object(
                binary_resource(
                    static_cast<void const*>(begin_sym),
                    static_cast<void const*>(end_sym)));
        }

        return nullptr;
    } else {
        /* not consteval, abort */
        return nullptr;
    }
}

constexpr resource* resource_loader::lookup_resource(std::string_view resource_name) {
    if constexpr {
        std::string begin_name = make_symbol_name(std::string(resource_name) + "_start");

        return const_cast<resource*>(get_by_symbol_reflection(begin_name));
    } else {
        return lookup_resource_runtime(resource_name);
    }
}
#endif
}

```

indexed-storage.h

```
#pragma once

#include <vector>
#include <cstdint>
#include <cstdint>
#include <cassert>
#include <utility>
#include <iterator>

namespace util {
    template<typename T>
    class indexed_storage {
    public:
        typedef std::uint32_t id_t;

        static constexpr id_t nullid = 0;

        struct entry {
            id_t id;
            T& value;
        };

        struct const_entry {
            id_t id;
            T const& value;
        };

        class iterator {
        private:
            indexed_storage* storage;
            size_t index;
            iterator(indexed_storage* st, size_t idx) : storage(st), index(idx) {}
            friend class indexed_storage<T>;
        public:
            using iterator_category = std::forward_iterator_tag;
            using value_type = entry;
            using difference_type = std::ptrdiff_t;
            using pointer = entry*;
            using reference = entry;

            iterator() : storage(nullptr), index(0) {}

            entry operator*() const {
                return {storage->lookup[index], storage->objects[index]};
            }

            iterator& operator++() { ++index; return *this; }
            iterator operator++(int) { iterator tmp = *this; ++(*this); return tmp; }
            iterator& operator--() { --index; return *this; }
            iterator operator--(int) { iterator tmp = *this; --(*this); return tmp; }

            bool operator==(iterator const& other) const { return index == other.index; }
            bool operator!=(iterator const& other) const { return index != other.index; }
        };

        class const_iterator {
```



```

private:
    indexed_storage const* storage;
    size_t index;
    const_iterator(indexed_storage const* st, size_t idx) : storage(st), index(idx) {}
    friend class indexed_storage<T>;
public:
    using iterator_category = std::forward_iterator_tag;
    using value_type        = const_entry;
    using difference_type    = std::ptrdiff_t;
    using pointer            = const_entry*;
    using reference          = const_entry;

    const_iterator() : storage(nullptr), index(0) {}

    const_entry operator*() const {
        return {storage->lookup[index], storage->objects[index]};
    }

    const_iterator& operator++() { ++index; return *this; }
    const_iterator operator++(int) { const_iterator tmp = *this; ++(*this); return tmp; }
    const_iterator& operator--() { --index; return *this; }
    const_iterator operator--(int) { const_iterator tmp = *this; --(*this); return tmp; }

    bool operator==(const_iterator const& other) const { return index == other.index; }
    bool operator!=(const_iterator const& other) const { return index != other.index; }
};

protected:
    std::vector<T> objects;
    std::vector<id_t> lookup;
    std::vector<size_t> reverse;

    id_t next_id = 1;

private:
    void init_from_objects() {
        size_t n = objects.size();
        lookup.reserve(n);
        reverse.reserve(n + 1);

        reverse.push_back(0);

        for (size_t i = 0; i < n; ++i) {
            id_t new_id = next_id++;
            lookup.push_back(new_id);
            reverse.push_back(i);
        }
    }

public:
    indexed_storage() {
        reverse.push_back(0);
    }

    explicit indexed_storage(std::vector<T> const& initial_objects)
        : objects(initial_objects) {
        init_from_objects();
    }

```

```

explicit indexed_storage(std::vector<T>&& initial_objects) noexcept
: objects(std::move(initial_objects)) {
    init_from_objects();
}

id_t size() const { return static_cast<id_t>(objects.size()); }

id_t add(T const& object) {
    id_t new_id = next_id++;

    if (new_id >= reverse.size()) {
        reverse.resize(new_id + 1, 0);
    }

    reverse[new_id] = objects.size();
    objects.push_back(object);
    lookup.push_back(new_id);

    return new_id;
}

id_t add(T&& object) {
    id_t new_id = next_id++;

    if (new_id >= reverse.size()) {
        reverse.resize(new_id + 1, 0);
    }

    reverse[new_id] = objects.size();
    objects.push_back(std::move(object));
    lookup.push_back(new_id);

    return new_id;
}

void remove(id_t id) {
    if (id == nullid || id >= reverse.size() || (reverse[id] == 0 && (objects.empty() || loo
        return;
    }

    size_t index_to_remove = reverse[id];
    size_t last_index = objects.size() - 1;

    if (index_to_remove != last_index) {
        T& last_obj = objects.back();
        id_t last_id = lookup.back();

        objects[index_to_remove] = std::move(last_obj);
        lookup[index_to_remove] = last_id;

        reverse[last_id] = index_to_remove;
    }

    objects.pop_back();
    lookup.pop_back();
    reverse[id] = 0;
}

```

```

T& operator[](id_t id) {
    assert(id != nullid && id < reverse.size() && reverse[id] < objects.size());
    return objects[reverse[id]];
}

T const& operator[](id_t id) const {
    assert(id != nullid && id < reverse.size() && reverse[id] < objects.size());
    return objects[reverse[id]];
}

iterator begin() { return iterator(this, 0); }
iterator end() { return iterator(this, objects.size()); }

const_iterator begin() const { return const_iterator(this, 0); }
const_iterator end() const { return const_iterator(this, objects.size()); }

const_iterator cbegin() const { return const_iterator(this, 0); }
const_iterator cend() const { return const_iterator(this, objects.size()); }

iterator erase(iterator it) {
    if (it != end()) {
        remove((*it).id);
    }
    return it;
}

const_iterator erase(const_iterator it) {
    if (it != cend()) {
        const_cast<indexed_storage*>(this)->remove((*it).id);
    }
    return it;
}

bool contains(id_t id) const {
    return id != nullid && id < reverse.size() &&
        (reverse[id] != 0 || (!objects.empty() && lookup[0] == id));
}
};
}

```

game.h

```
#pragma once

#include <vector>
#include <memory>

#include "util/resource-loader.h"
#include "assets/asset-manager.h"
#include "input/input-backend.h"
#include "rendering/rendering-backend.h"
#include "audio/audio-backend.h"
#include "geometry/map-data.h"
#include "engine/world.h"
#include "audio/audio-mixer.h"
#include "rendering/renderer-2d.h"
#include "rendering/software-renderer.h"
#include "entities/player.h"

namespace game {
    class game {
        util::resource_loader rl;
        assets::asset_manager am;
        std::unique_ptr<input::input_backend> input;
        std::unique_ptr<rendering::rendering_backend> rb;
        std::unique_ptr<audio::audio_backend> ab;
        geometry::map_data md;
        audio::audio_mixer mix;
        rendering::renderer_2d r2d;
        rendering::software_renderer sr;
        engine::world w;
        entities::player* player_ptr;

        float fov;

        void text_scroll(std::vector<std::string> const& text, assets::audio_clip_id const aid);
        void ts_from_resource(std::string const& res_name, assets::audio_clip_id const aid);
        void opening();
        void ending();
        void credits();
        void show_main_menu();
        void show_options();
        void loop();
        void draw_hud();
    public:
        game();
        void run();
    };
}
```

subsector.h

```
#pragma once

#include "util/resource.h"
#include "util/indexed-storage.h"
#include "rendering/sprite.h"
#include <vector>

namespace rendering {
    class software_renderer;
}

namespace geometry {
    class linedef;

    class subsector {
        std::vector<util::indexed_storage<linedef>::id_t> lines;
        std::vector<rendering::sprite*> sprites;

    public:

        subsector() = default;
        subsector(std::vector<util::indexed_storage<linedef>::id_t> l);

        static std::vector<subsector> load_from_bin(util::resource const& res);

        void remove_sprite(rendering::sprite* spr);
        void add_sprite(rendering::sprite* spr);

        friend rendering::software_renderer;
    };
}
```

sector.h

```
#pragma once

#include "assets/asset-manager.h"
#include "util/resource.h"
#include <vector>
#include <cstdint>

namespace rendering {
    class software_renderer;
}

namespace geometry {
    class sector {
        float floor_height;
        float ceiling_height;
        assets::texture_id floor_tex;
        assets::texture_id ceiling_tex;
        std::uint8_t light_level;

    public:

        sector(float fh, float ch, assets::texture_id ft, assets::texture_id ct, std::uint8_t light)

        static std::vector<sector> load_from_bin(util::resource const& res);

        friend rendering::software_renderer;
    };
}
```

monster-spawn.h

```
#pragma once

#include <vector>
#include <cstdint>
#include "math/vec2.h"
#include "util/resource.h"
#include "util/componentized.h"
#include "entities/monster-factory.h"

namespace geometry {
    class monster_spawn : public util::componentized<monster_spawn> {
        std::uint32_t type;
        [[=util::component_field{}}} math::vec2 pos;
        float z;

        friend class util::componentized<monster_spawn>;

    public:
        monster_spawn(std::uint32_t t, math::vec2 p, float height);

        static std::vector<monster_spawn> load_from_bin(util::resource const& res);

        friend std::unique_ptr<entities::monster> entities::make_monster(geometry::monster_spawn const& spawn);
    };
}
```

sidedef.h

```
#pragma once

#include "assets/ids.h"
#include "util/resource.h"
#include "util/indexed-storage.h"
#include <vector>

namespace rendering {
    class software_renderer;
}

namespace geometry {
    class sector;

    class sidedef {
        util::indexed_storage<sector>::id_t facing_sector;
        assets::texture_id upper_tex;
        assets::texture_id middle_tex;
        assets::texture_id lower_tex;

    public:

        sidedef(util::indexed_storage<sector>::id_t sector, assets::texture_id ut, assets::texture_id mt, assets::texture_id lt) :
            facing_sector(sector), upper_tex(ut), middle_tex(mt), lower_tex(lt) {}

        static std::vector<sidedef> load_from_bin(util::resource const& res);

        friend rendering::software_renderer;
    };
}
```


bsp-node.h

```
#pragma once

#include "math/vec2.h"
#include "util/indexed-storage.h"
#include "util/resource.h"
#include <vector>

namespace rendering {
    class software_renderer;
}

namespace geometry {
    class bsp_node {
    public:
        struct bounding_box {
            float top, bottom, left, right;
        };

    private:
        math::vec2 pl_coord, pl_dir;
        bounding_box front_box, back_box;
        util::indexed_storage<bsp_node>::id_t front, back;
        static constexpr util::indexed_storage<bsp_node>::id_t leaf_flag = 1 << (sizeof(util::indexe

    public:

        __attribute__((always_inline)) inline constexpr bool is_front_leaf() const;
        __attribute__((always_inline)) inline constexpr bool is_back_leaf() const;
        __attribute__((always_inline)) inline constexpr util::indexed_storage<bsp_node>::id_t get_fr
        __attribute__((always_inline)) inline constexpr util::indexed_storage<bsp_node>::id_t get_ba

        bool is_pt_front_side(math::vec2 const& point) const;

        bsp_node(math::vec2 const pc, math::vec2 const pd, bounding_box const fb, bounding_box const
        static std::vector<bsp_node> const load_from_bin(util::resource const& res);

        __attribute__((always_inline)) static inline constexpr bool is_leaf(util::indexed_storage<bs
        __attribute__((always_inline)) static inline constexpr util::indexed_storage<bsp_node>::id_t

        friend rendering::software_renderer;
    };
}

__attribute__((always_inline)) inline constexpr bool geometry::bsp_node::is_front_leaf() const {
    return is_leaf(front);
}

__attribute__((always_inline)) inline constexpr bool geometry::bsp_node::is_back_leaf() const {
    return is_leaf(back);
}

__attribute__((always_inline)) inline constexpr util::indexed_storage<geometry::bsp_node>::id_t geom
    return get_id(front);
}
```

```

__attribute__((always_inline)) inline constexpr util::indexed_storage<geometry::bsp_node>::id_t geom
    return get_id(back);
}

__attribute__((always_inline)) inline constexpr bool geometry::bsp_node::is_leaf(util::indexed_stora
    return id & geometry::bsp_node::leaf_flag;
}

__attribute__((always_inline)) inline constexpr util::indexed_storage<geometry::bsp_node>::id_t geom
    return raw & ~geometry::bsp_node::leaf_flag;
}

```

linedef.h

```
#pragma once
```

```
#include "math/segment.h"
#include "util/resource.h"
#include "util/indexed-storage.h"
#include "util/componentized.h"
#include <vector>
```

```
namespace rendering {
    class software_renderer;
}
```

```
namespace geometry {
    class sidedef;
```

```
    class linedef : public util::componentized<linedef> {
        [[=util::component_field{}]] math::segment seg;
```

```
        util::indexed_storage<sidedef>::id_t front;
        util::indexed_storage<sidedef>::id_t back;
```

```
    public:
```

```
        linedef(math::vec2 const p0, math::vec2 const p1, util::indexed_storage<sidedef>::id_t const
```

```
        static std::vector<linedef> load_from_bin(util::resource const& res);
```

```
        __attribute__((always_inline)) constexpr inline bool is_wall() const;
        __attribute__((always_inline)) constexpr inline bool is_portal() const;
        __attribute__((always_inline)) constexpr inline math::vec2 dir() const;
        __attribute__((always_inline)) constexpr inline float len() const;
```

```
        friend util::componentized<linedef>;
        friend rendering::software_renderer;
```

```
    };
```

```
}
```

```
__attribute__((always_inline)) constexpr inline bool geometry::linedef::is_wall() const {
    return back == util::indexed_storage<sidedef>::nullid;
}
```

```
}
```

```
__attribute__((always_inline)) constexpr inline bool geometry::linedef::is_portal() const {
    return !is_wall();
}
```

```
}
```

```
__attribute__((always_inline)) constexpr inline math::vec2 geometry::linedef::dir() const {
    return (seg("point1"_f) - seg("point0"_f)).normalized();
}
```

```
}
```

```
__attribute__((always_inline)) constexpr inline float geometry::linedef::len() const {
    return seg.len();
}
```

```
}
```

pickup-spawn.h

```
#pragma once

#include <vector>
#include <cstdint>
#include <memory>
#include "math/vec2.h"
#include "util/resource.h"
#include "util/componentized.h"
#include "world_objects/pickup.h"
#include "assets/asset-manager.h"

namespace entities { class player; }
namespace audio { class audio_mixer; }
namespace engine { class world; }
namespace geometry { class pickup_spawn; }
namespace world_object {
    std::unique_ptr<world_object::pickup> make_pickup(geometry::pickup_spawn const& ps, entities::pl
}

namespace geometry {
    class pickup_spawn : public util::componentized<pickup_spawn> {
        std::uint32_t type;
        std::uint32_t subtype;
        [[=util::component_field{}}} math::vec2 pos;
        float z;

        friend class util::componentized<pickup_spawn>;

    public:
        pickup_spawn(std::uint32_t t, std::uint32_t st, math::vec2 p, float height);

        static std::vector<pickup_spawn> load_from_bin(util::resource const& res);

        friend std::unique_ptr<world_object::pickup> world_object::make_pickup(pickup_spawn const& p
    };
}
```

map-data.h

```
#pragma once
```

```
#include "sector.h"
#include "sidedef.h"
#include "linedef.h"
#include "subsector.h"
#include "bsp-node.h"
#include "monster-spawn.h"
#include "pickup-spawn.h"
#include "rendering/sprite.h"
#include "util/indexed-storage.h"
#include "util/componentized.h"
```

```
namespace rendering {
    class software_renderer;
}
namespace game {
    class game;
}
```

```
namespace geometry {
    class map_data : public util::componentized<map_data> {
        util::indexed_storage<sector> sectors;
        util::indexed_storage<sidedef> sidedefs;
        [[=util::component_field{}]] util::indexed_storage<linedef> linedefs;
        [[=util::ref_component_field{}]] util::indexed_storage<subsector> subsectors;
        util::indexed_storage<bsp_node> nodes;
        std::vector<monster_spawn> monster_spawns;
        std::vector<pickup_spawn> pickup_spawns;

        [[=util::component_field{}]] util::indexed_storage<bsp_node>::id_t root_node_id;

    public:

        util::indexed_storage<subsector>::id_t get_subsector_id(math::vec2 const& pt) const;

        void move_to(rendering::sprite* spr, math::vec2 const& new_pos);

        static map_data load_from_bin(util::resource const& sectors_res, util::resource const& sidedefs_res);

        friend util::componentized<map_data>;
        friend rendering::software_renderer;
        friend game::game;
    };
}
```

ids.h

```
#pragma once

#include <cstdint>

namespace assets {
    typedef std::int32_t texture_id;
    typedef std::int32_t menu_id;
    typedef std::int32_t audio_clip_id;
}
```

audio-clip.h

```
#pragma once
#define AUDIO_CLIP_H

#include <vector>
#include <cstdint>
#include "util/resource.h"
#include "util/componentized.h"
#include "audio/audio-backend.h"

namespace assets {
    class audio_clip : public util::componentized<audio_clip> {
        audio_clip(std::vector<std::uint8_t> const &data, audio::audio_format const &fmt);

        [[=util::component_field{}]] std::vector<std::uint8_t> pcm_data;
        [[=util::component_field{}]] audio::audio_format format;
        [[=util::component_field{}]] unsigned long total_frames;

    public:
        static audio_clip const load_from_bin(util::resource const &res);

        friend util::componentized<audio_clip>;
    };
}
```

menu.h

```
#pragma once

#include <vector>
#include <string>
#include <string_view>
#include <functional>

#include "util/resource.h"
#include "util/resource-loader.h"
#include "texture.h"
#include "menu-element.h"
#include "input/input-backend.h"
#include "ids.h"

namespace rendering {
    class rendering_backend;
    class render_2d;
}

namespace assets {
    class menu {
        menu(std::vector<menu_element> els, texture_id const b);

        std::vector<menu_element> elements;
        texture_id const bg;

    public:
        static menu load_from_bin(util::resource_loader &resld, util::resource const &res);

        /* usage: */
        /* menu.set_formatter_for("{name}", [&](std::string_view tpl) { */
        /*     std::string s(tpl); */
        /*     for (std::size_t p; (p = s.find("{name}")) != std::string::npos;) */
        /*         s.replace(p, 7, player_name); */
        /*     return s; */
        /* }); */
        void selective_formatter(std::string_view key_filter, std::function<std::string(std::string_view)> fn);

        void formatter_all(std::function<std::string(std::string_view)> fn);

        int display(rendering::render_2d const &r2d, rendering::rendering_backend &rb, input::input_backend &ib);
    };
}
```


menu-element.h

```
#pragma once

#include <vector>
#include <string>
#include <functional>

#include "util/componentized.h"
#include "util/resource.h"
#include "texture.h"
#include "ids.h"

namespace rendering {
    class renderer_2d;
}

namespace input {
    struct mouse_state;
}

namespace assets {
    class menu_element {
        menu_element(int f, float bx, float by, float ex, float ey, texture_id const b, std::string

        int function;
        float const begin_x, begin_y, end_x, end_y;
        texture_id const bg;
        std::string fmt_template;
        std::function<std::string(std::string_view)> formatter;
        int char_w, char_h;
        std::uint8_t col_r, col_g, col_b;
        float relative_left_margin;

        std::string resolve_text() const;
        std::uint32_t text_color() const;
        void selective_fmt(std::string_view key_filter, std::function<std::string(std::string_view)>
        int render(rendering::renderer_2d const &r2d, std::vector<texture> const& ui_tx, int sw, int

    public:
        static menu_element load_from_bin(util::resource const &res);

        friend class menu;
    };
}
```

asset-manager.h

```
#pragma once

#include <vector>
#include <string>
#include <string_view>

#include "util/resource.h"
#include "util/resource-loader.h"
#include "asset-pack.h"
#include "input/input-backend.h"
#include "ids.h"

namespace rendering {
    class renderer_2d;
    class rendering_backend;
}

namespace assets {
    class asset_manager {
        util::resource_loader &rl;
        std::vector<asset_pack> asset_packs;
        unsigned int cur_set;

        asset_manager(util::resource_loader &resld, std::vector<asset_pack> packs);

    public:
        static asset_manager load(util::resource_loader &resld);

        void cycle_set();

        texture const& wall_tx_by_id(texture_id const id) const;
        texture const& sprite_tx_by_id(texture_id const id) const;
        texture const& flat_tx_by_id(texture_id const id) const;
        texture const& ui_tx_by_id(texture_id const id) const;
        menu &menu_by_id(menu_id const id);
        audio_clip const& audio_clip_by_id(audio_clip_id const id) const;

        int display_menu(menu_id const id, rendering::renderer_2d const& r2d, rendering::rendering_b
    };
}
```

texture.h

```
#pragma once

#include <vector>
#include <cstdint>
#include <cstdint>

#include "util/resource.h"
#include "util/componentized.h"

namespace assets {
    class texture : public util::componentized<texture> {
        texture(std::vector<std::uint32_t> const &t, std::uint32_t const w, std::uint32_t const h, b

        [[=util::component_field{}}} std::vector<std::uint32_t> pixels;
        [[=util::component_field{}}} std::uint32_t width, height;

        [[=util::component_field{}}} bool has_transparency;

    public:
        static texture const load_from_bin(util::resource const &res);

        friend util::componentized<texture>;
    };
}
```

asset-pack.h

```
#pragma once

#include <vector>
#include <cstdint>
#include <string_view>
#include <unordered_map>

#include "util/resource.h"
#include "util/resource-loader.h"
#include "texture.h"
#include "menu-element.h"
#include "menu.h"
#include "audio-clip.h"
#include "ids.h"

namespace assets {
    class asset_pack {
        std::vector<texture> wall_textures;
        std::vector<texture> sprite_textures;
        std::vector<texture> flat_textures;
        std::vector<texture> ui_textures;
        std::vector<menu> menus;
        std::vector<audio_clip> audio_clips;

        asset_pack(std::vector<texture> walls, std::vector<texture> sprites, std::vector<texture> flat_textures,
            std::vector<texture> ui_textures, std::vector<menu> menus, std::vector<audio_clip> audio_clips) :
            wall_textures(walls), sprite_textures(sprites), flat_textures(flat_textures), ui_textures(ui_textures),
            menus(menus), audio_clips(audio_clips) {}

        static std::vector<texture> tx_from_meta(util::resource_loader &resld, std::string_view meta_path);
        static std::vector<menu> menus_from_meta(util::resource_loader &resld, std::string_view meta_path);
        static std::vector<audio_clip> clips_from_meta(util::resource_loader &resld, std::string_view meta_path);
        static std::unordered_map<std::string, std::string> parse_meta_keys(std::string const& content);

    public:
        static asset_pack load(util::resource_loader &resld, std::string_view pack_meta_path);

        texture const& wall_tx_by_id(texture_id const id) const;
        texture const& sprite_tx_by_id(texture_id const id) const;
        texture const& flat_tx_by_id(texture_id const id) const;
        texture const& ui_tx_by_id(texture_id const id) const;
        menu &menu_by_id(menu_id const id);
        audio_clip const& audio_clip_by_id(audio_clip_id const id) const;

        int display_menu(menu_id const id, rendering::renderer_2d const& r2d, rendering::camera const& cam);
    };
}
```

health_systems.h

```
#pragma once

#include <vector>
#include <memory>
#include <algorithm>

#include "combat/status-effect.h"
#include "util/indexed-storage.h"
#include "util/componentized.h"

namespace engine {class actor;}

namespace systems {
    class health_system : public util::componentized<health_system> {
        [=util::component_field{}]] float current_hp;
        [=util::component_field{}]] float max_hp;
        [=util::component_field{}]] float armor;
        [=util::component_field{}]] float max_armor;

        [=util::component_field{}]] std::vector<std::unique_ptr<combat::status_effect>> active_effects;

        friend class util::componentized<health_system>;

    public:
        health_system(float hp, float max, float arm, float max_arm);

        void apply_damage(float amount);
        void apply_heal(float amount);
        void apply_shield(float amount);
        void apply_true_damage(float amount);

        void add_effect(std::unique_ptr<combat::status_effect> effect, engine::actor& owner);
        void process_effects(float dt, engine::actor& owner);

        bool is_dead() const;
    };
}
```

backend.h

```
#pragma once

#include <alsa/asoundlib.h>
#include "audio/audio-backend.h"

namespace audio {
    namespace alsa {
        class backend : public audio::audio_backend {
        private:
            bool is_bad;

            snd_pcm_t* pcm_handle;
            audio_format fmt;
            bool paused;

            snd_pcm_format_t to_alsa_format(unsigned int bits) const;

            bool configure_hw_params();

        public:
            backend();
            ~backend() override;

            bool open(audio_format const& format) override;
            void close() override;
            long write(void const* data, unsigned long frames) override;
            void drain() override;
            void pause() override;
            void resume() override;
            audio_format operator()(util::component_tag<"current_format">) const override;
            bool bad() const override;
        };
    }
}
```

audio-mixer.h

```
#pragma once
#define AUDIO_MIXER_H

#include "audio-backend.h"
#include "assets/audio-clip.h"
#include <vector>

namespace audio {

    class audio_mixer {
    private:
        struct playing_sound {
            assets::audio_clip const* clip;
            unsigned long current_frame;
        };

        audio_backend &target;
        audio_format mix_format;
        std::vector<playing_sound> active_sounds;

    public:
        explicit audio_mixer(audio_backend &tgt, audio_format const& fmt);

        void play(assets::audio_clip const& clip);

        void stop_all();

        void step(unsigned long frames_to_mix);

        bool is_silent() const;
    };
}...
```

audio-backend.h

```
#pragma once

#include <cstddef>
#include <cstdint>
#include <string>
#include <vector>

#include "util/componentized.h"

namespace audio {

    struct audio_format {
        unsigned int sample_rate;
        unsigned int channels;
        unsigned int bits_per_sample;
    };

    class audio_backend {
    public:
        virtual ~audio_backend() = default;

        virtual bool open(audio_format const& fmt) = 0;
        virtual void close() = 0;
        virtual long write(void const* data, unsigned long frames) = 0;
        virtual void drain() = 0;

        virtual void pause() = 0;
        virtual void resume() = 0;
        virtual audio_format operator()(util::component_tag<"current_format">) const = 0;
        virtual bool bad() const = 0;
    };
}
```


actor.h

```
#pragma once

#include "renderable-entity.h"
#include "systems/health_systems.h"
#include "util/componentized.h"

namespace combat { class charmed; class slowed; }

namespace engine {

    enum class faction {
        player,
        enemy,
        neutral
    };

    class actor : public renderable_entity, public util::componentized<actor> {
    public:
        using util::componentized<actor>::operator();
        using rendering::sprite::operator();
    protected:
        [[=util::component_field{}}} systems::health_system health;
        float movement_speed;
        faction team;

        friend class util::componentized<actor>;
        friend class combat::charmed;
        friend class combat::slowed;
        friend class projectile;

    public:
        actor(math::vec2 const p, float const z, assets::texture_id const tex, float const is, float

        virtual void take_damage(float const dmg);
        virtual void take_true_damage(float const dmg);
        virtual void heal(float const amount);
        virtual void add_shield(float const amount);
        virtual void add_effect(std::unique_ptr<combat::status_effect> effect);

        bool is_dead() const;

        virtual void update(float dt) override;
    };
}
```

entity.h

```
#pragma once

namespace engine {
    class entity {
    public:
        virtual void update(float const dt) = 0;
        virtual ~entity() = default;
    };
}...
```

projectile.h

```
#pragma once

#include "renderable-entity.h"
#include "actor.h"
#include "world.h"
#include "math/vec2.h"
#include "combat/status-effect.h"
#include <memory>
#include <functional>

namespace geometry { class map_data; }
namespace combat { namespace weapons { class projectile_firing_mode; } }
namespace entities { class monster; class monster_boss; }

namespace engine {

    class projectile : public renderable_entity {
        friend class combat::weapons::projectile_firing_mode;
        friend class entities::monster;
        friend class entities::monster_boss;
    protected:
        math::vec2 direction;
        float speed;
        float damage;
        float lifetime;
        float hit_radius;
        faction team;
        std::function<std::unique_ptr<combat::status_effect>()> on_hit_effect;

        world* world_ref = nullptr;
        geometry::map_data* map_ref = nullptr;
        util::indexed_storage<std::unique_ptr<entity>>::id_t self_id = 0;

    public:
        projectile(math::vec2 pos, float z, assets::texture_id tex, float scale,
                   math::vec2 dir, float spd, float dmg, float life,
                   faction f, float radius = 8.0f);

        void update(float dt) override;
    };
}
```

world.h

```
#pragma once
#include <vector>
#include <memory>

#include "util/indexed-storage.h"
#include "entity.h"

namespace engine {
    class world {
    private:
        util::indexed_storage<std::unique_ptr<entity>> entities;
        std::vector<util::indexed_storage<std::unique_ptr<entity>>::id_t> deleted_entities;

    public:

        world();

        world(const world&) = delete;
        world& operator=(const world&) = delete;

        void update(float const dt);

        entity& operator[] (util::indexed_storage<std::unique_ptr<entity>>::id_t const id);
        entity const& operator[] (util::indexed_storage<std::unique_ptr<entity>>::id_t const id) const;

        util::indexed_storage<std::unique_ptr<entity>> const& get_entities() const;

        util::indexed_storage<std::unique_ptr<entity>>::id_t register_entity(std::unique_ptr<entity>)

        void delete_entity(util::indexed_storage<std::unique_ptr<entity>>::id_t const id);

        entity* entity_from_id(util::indexed_storage<std::unique_ptr<entity>>::id_t const id) const;
    };
}
```

renderable-entity.h

```
#pragma once
#include "entity.h"
#include "rendering/sprite.h"

namespace engine {
    class renderable_entity : public entity, public rendering::sprite {
    public:
        renderable_entity(math::vec2 const p, float const z, assets::texture_id const tex, float const is)
            : entity(), rendering::sprite(p, z, tex, is) {}

        //renderable_entity() : rendering::sprite() {}

        virtual ~renderable_entity() = default;
    };
}
```

pickup-factory.h

```
#pragma once

#include <memory>
#include <cstdint>
#include "math/vec2.h"
#include "world_objects/pickup.h"

namespace geometry {
    class map_data;
    class pickup_spawn;
}

namespace engine { class world; }
namespace entities { class player; }
namespace audio { class audio_mixer; }
namespace assets { class asset_manager; }

namespace world_object {
    std::unique_ptr<pickup> make_pickup(geometry::pickup_spawn const& ps, entities::player& player,
}
```

pickup.h

```
#pragma once

#include "engine/renderable-entity.h"
#include "math/vec2.h"
#include "util/indexed-storage.h"
#include "geometry/subsector.h"

namespace entities { class player; }
namespace geometry { class map_data; }

namespace world_object {

    class pickup : public engine::renderable_entity {
        float pickup_radius;
        entities::player& player_ref;
        geometry::map_data& map_ref;
        util::indexed_storage<geometry::subsector>::id_t subsector_id;
        bool consumed = false;

    public:
        pickup(math::vec2 pos, float z, assets::texture_id tex,
            entities::player& p, geometry::map_data& md,
            util::indexed_storage<geometry::subsector>::id_t sub_id,
            float radius = 20.0f, float scale = 1.0f);

        void update(float dt) override;
        bool is_consumed() const;

        virtual void on_pickup(entities::player& p) = 0;
        virtual ~pickup() = default;
    };
}
```

weapon-pickup.h

```
#pragma once

#include <memory>
#include "pickup.h"

namespace combat { namespace weapons { class weapon; } }

namespace world_object {

    class weapon_pickup : public pickup {
        std::unique_ptr<combat::weapons::weapon> provided_weapon;
        int slot;
    public:
        weapon_pickup(math::vec2 pos, float z, assets::texture_id tex,
                      std::unique_ptr<combat::weapons::weapon> w, int weapon_slot,
                      entities::player& p, geometry::map_data& md,
                      util::indexed_storage<geometry::subsector>::id_t sub_id,
                      float radius = 25.0f, float scale = 0.4f);
        void on_pickup(entities::player& p) override;
    };

}
```


armor-pickup.h

```
#pragma once
```

```
#include "pickup.h"
```

```
namespace world_object {
```

```
    class armor_pickup : public pickup {
```

```
        float armor_amount;
```

```
    public:
```

```
        armor_pickup(math::vec2 pos, float z, assets::texture_id tex,
```

```
                    float amount, entities::player& p,
```

```
                    geometry::map_data& md,
```

```
                    util::indexed_storage<geometry::subsector>::id_t sub_id,
```

```
                    float radius = 20.0f);
```

```
        void on_pickup(entities::player& p) override;
```

```
    };
```

```
}
```

health-pickup.h

```
#pragma once

#include "pickup.h"

namespace world_object {

    class health_pickup : public pickup {
        float heal_amount;
    public:
        health_pickup(math::vec2 pos, float z, assets::texture_id tex,
            float amount, entities::player& p,
            geometry::map_data& md,
            util::indexed_storage<geometry::subsector>::id_t sub_id,
            float radius = 20.0f);
        void on_pickup(entities::player& p) override;
    };

}
```

ammo-pickup.h

```
#pragma once

#include "pickup.h"
#include "entities/player.h"

namespace world_object {

    template<typename WeaponT>
    class ammo_pickup : public pickup {
        int amount;
    public:
        ammo_pickup(math::vec2 position, float z, assets::texture_id tex,
                    int amt, entities::player& pl,
                    geometry::map_data& md,
                    util::indexed_storage<geometry::subsector>::id_t sub_id,
                    float radius = 20.0f)
            : pickup(position, z, tex, pl, md, sub_id, radius), amount(amt) {}

        void on_pickup(entities::player& p) override {
            p.resupply<WeaponT>(amount);
        }
    };
}
```

burning.h

```
#pragma once

#include "status-effect.h"

namespace combat
{
    class burning : public status_effect {
    public:
        burning(float const dur, unsigned int intens);

        void affect(engine::actor& target) override;
    };
}
```

charmed.h

```
#pragma once

#include "status-effect.h"

namespace combat
{
    class charmed : public status_effect {
    public:
        charmed(float const dur, unsigned int intens);

        void on_apply(engine::actor& target) override;
        void on_expire(engine::actor& target) override;
    };
}
```

slowed.h

```
#pragma once

#include "status-effect.h"

namespace combat
{
    class slowed : public status_effect {
    public:
        slowed(float const dur, unsigned int intens);

        void on_apply(engine::actor& target) override;
        void on_expire(engine::actor& target) override;
    };
}
```

firing-mode.h

```
#pragma once

#include "math/vec2.h"

namespace combat {
    namespace weapons {
        class firing_mode {
        public:
            virtual void spawn_bullet(math::vec2 pos, float angle, float damage) = 0;
            virtual ~firing_mode() = default;
        };
    }
}
```

smg.h

```
#pragma once
#include "weapon.h"

namespace geometry { class map_data; }
namespace engine { class world; }

namespace combat {
    namespace weapons {
        class smg : public weapon {
        public:
            explicit smg(geometry::map_data const& map,
                        engine::world const& world,
                        audio::audio_mixer& mix,
                        assets::asset_manager const& am);
        };
    }
}
```


projectile-firing-mode.h

```
#pragma once

#include "firing-mode.h"
#include "engine/actor.h"
#include "assets/ids.h"

namespace engine { class world; }
namespace geometry { class map_data; }

namespace combat {
    namespace weapons {

        class projectile_firing_mode : public firing_mode {
            engine::world& world_ref;
            geometry::map_data& map_ref;
            engine::faction team;
            assets::texture_id projectile_tex;
            float projectile_speed;
            float projectile_lifetime;
            float projectile_scale;

        public:
            projectile_firing_mode(engine::world& w, geometry::map_data& md,
                                   engine::faction f,
                                   assets::texture_id tex, float spd,
                                   float lifetime = 5.0f, float scale = 0.5f);

            void spawn_bullet(math::vec2 pos, float angle, float damage) override;
        };
    }
}
```

katana.h

```
#pragma once
#include "weapon.h"

namespace geometry { class map_data; }
namespace engine { class world; }

namespace combat {
    namespace weapons {
        class katana : public weapon {
        public:
            katana(geometry::map_data const& map, engine::world const& world,
                  audio::audio_mixer& mix, assets::asset_manager const& am);

            bool can_fire() const override;
            void fire(math::vec2 pos, float angle) override;
            void reload() override;
        };
    }
}
```

hitscan-firing-mode.h

```
#pragma once

#include "firing-mode.h"
#include "geometry/map-data.h"
#include "engine/actor.h"

namespace engine { class world; }

namespace combat {
    namespace weapons {
        class hitscan_firing_mode : public firing_mode {
            geometry::map_data const& map;
            engine::world const& world_ref;
            float max_range;
            float hit_radius;

        public:
            hitscan_firing_mode(geometry::map_data const& map,
                               engine::world const& world,
                               float range = 8192.0f,
                               float radius = 32.0f);

            void spawn_bullet(math::vec2 pos, float angle, float damage) override;
        };
    }
}
```

pistol.h

```
#pragma once
#include "weapon.h"

namespace geometry { class map_data; }
namespace engine { class world; }

namespace combat {
    namespace weapons {
        class pistol : public weapon {
        public:
            explicit pistol(geometry::map_data const& map,
                           engine::world const& world,
                           audio::audio_mixer& mix,
                           assets::asset_manager const& am);
        };
    }
}
```

shotgun.h

```
#pragma once
#include "weapon.h"

namespace geometry { class map_data; }
namespace engine { class world; }

namespace combat {
    namespace weapons {
        class shotgun : public weapon {
        public:
            static constexpr int    pellet_count = 8;
            static constexpr float  spread      = 0.2618f;

            explicit shotgun(geometry::map_data const& map,
                             engine::world const& world,
                             audio::audio_mixer& mix,
                             assets::asset_manager const& am);
            void fire(math::vec2 pos, float angle) override;
        };
    }
}
```

rifle.h

```
#pragma once
#include "weapon.h"

namespace geometry { class map_data; }
namespace engine { class world; }

namespace combat {
    namespace weapons {
        class rifle : public weapon {
        public:
            explicit rifle(geometry::map_data const& map,
                           engine::world const& world,
                           audio::audio_mixer& mix,
                           assets::asset_manager const& am);
        };
    }
}
```

weapon.h

```
#pragma once

#include <memory>

#include "math/vec2.h"
#include "firing-mode.h"
#include "assets/ids.h"
#include "util/componentized.h"

namespace audio { class audio_mixer; }
namespace assets { class asset_manager; }

namespace combat
{
    namespace weapons {
        class weapon : public util::componentized<weapon> {
            friend class util::componentized<weapon>;
        protected:
            std::unique_ptr<firing_mode> ammo;
            [[=util::component_field{}]] int ammo_count;
            int max_ammo;
            [[=util::component_field{}]] int reserve_mags;
            float fire_rate;
            float last_shot_time;
            float damage;
            audio::audio_mixer& mixer;
            assets::asset_manager const& assets;
            assets::audio_clip_id fire_sound_id;
            assets::audio_clip_id reload_sound_id;
            [[=util::component_field{}]] float reload_duration;
            [[=util::component_field{}]] float reload_timer;
            [[=util::component_field{}]] bool reloading;

            void finish_reload();
        public:
            virtual bool can_fire() const;
            void tick(float dt);
            void resupply(int amount);

            virtual void fire(math::vec2 pos, float angle);
            virtual void reload();
            virtual ~weapon() = default;

        protected:
            weapon(std::unique_ptr<firing_mode> firing, int max, float rate, float dmg,
                int reserve,
                audio::audio_mixer& mix,
                assets::asset_manager const& am,
                assets::audio_clip_id fire_snd = -1,
                assets::audio_clip_id reload_snd = -1,
                float reload_dur = 1.5f);
    };
}
}
```

sniper-rifle.h

```
#pragma once
#include "weapon.h"

namespace geometry { class map_data; }
namespace engine { class world; }

namespace combat {
    namespace weapons {
        class sniper_rifle : public weapon {
        public:
            explicit sniper_rifle(geometry::map_data const& map,
                                engine::world const& world,
                                audio::audio_mixer& mix,
                                assets::asset_manager const& am);
        };
    }
}
```


plasma-gun.h

```
#pragma once
#include "weapon.h"

namespace geometry { class map_data; }
namespace engine { class world; }

namespace combat {
    namespace weapons {
        class plasma_gun : public weapon {
        public:
            explicit plasma_gun(geometry::map_data& map,
                               engine::world& world,
                               audio::audio_mixer& mix,
                               assets::asset_manager const& am);
        };
    }
}
```

status-effect.h

```
#pragma once

#include "math/vec2.h"

namespace engine {class actor;}

namespace combat
{
    class status_effect {
    protected:
        float duration;
        unsigned int intensity;
        float tick_interval;
        float tick_timer;
    public:
        status_effect(float const dur, float tick_inter, unsigned int intens);

        virtual void on_apply(engine::actor&);
        virtual void on_expire(engine::actor&);
        virtual void affect(engine::actor&);
        virtual ~status_effect() = default;

        bool tick(float dt);
        bool is_expired() const;
    };
}
```

monster-factory.h

```
#pragma once

#include <memory>
#include <cstdint>
#include "math/vec2.h"
#include "entities/monster.h"

namespace engine { class world; }

namespace geometry {
    class monster_spawn;
    class map_data;
}

namespace entities {
    std::unique_ptr<monster> make_monster(geometry::monster_spawn const& ms, engine::actor& target,
}
```

slug-projectile.h

```
#pragma once

#include "engine/projectile.h"

namespace entities {

    class slug_projectile : public engine::projectile {
    public:
        using projectile::projectile;
    };

}
```

monster-magic.h

```
#pragma once
```

```
#include "entities/monster.h"
```

```
namespace entities {
```

```
    class monster_magic : public monster {
```

```
        float charge_time;
```

```
        float charge_timer;
```

```
        bool is_charging;
```

```
        float post_fire_cooldown;
```

```
        float sidestep_timer;
```

```
        float sidestep_sign;
```

```
    public:
```

```
        monster_magic(math::vec2 const p, float const z, engine::actor& target, geometry::map_data&
```

```
        void update(float dt) override;
```

```
};
```

```
}
```

monster-elite-swift.h

```
#pragma once
```

```
#include "entities/monster.h"
```

```
namespace entities {
```

```
    class monster_elite_swift : public monster {
```

```
        float charge_speed;
```

```
        bool is_charging;
```

```
        float charge_timer;
```

```
        float charge_cd;
```

```
        float circle_angle;
```

```
        float circle_radius;
```

```
    public:
```

```
        monster_elite_swift(math::vec2 const p, float const z, engine::actor& target, geometry::map_
```

```
        void update(float dt) override;
```

```
};
```

```
}
```

monster-basic.h

```
#pragma once
```

```
#include "entities/monster.h"
```

```
namespace entities {
```

```
    class monster_basic : public monster {
```

```
    public:
```

```
        monster_basic(math::vec2 const p, float const z, engine::actor& target, geometry::map_data& map_data)
```

```
        void update(float dt) override;
```

```
    };
```

```
}
```

monster-trapper.h

```
#pragma once
```

```
#include "entities/monster.h"
```

```
namespace entities {
```

```
    class monster_trapper : public monster {
```

```
        int    max_traps;
```

```
        int    traps_placed;
```

```
        float  trap_timer;
```

```
        float  trap_interval;
```

```
        float  wander_timer;
```

```
        float  wander_sign;
```

```
    public:
```

```
        monster_trapper(math::vec2 const p, float const z, engine::actor& target, geometry::map_data
```

```
        void update(float dt) override;
```

```
};
```

```
}
```


monster-duzy-gruby.h

```
#pragma once
```

```
#include "entities/monster.h"
```

```
namespace entities {
```

```
    class monster_Duzy_Gruby : public monster {  
    public:
```

```
        monster_Duzy_Gruby(math::vec2 const p, float const z, engine::actor& target, geometry::map_d
```

```
        void update(float dt) override;
```

```
    };
```

```
}
```

monster-spawner.h

```
#pragma once
```

```
#include "entities/monster.h"
```

```
namespace entities {
```

```
    class monster_spawner : public monster {
```

```
        int    max_spawns;
```

```
        int    current_spawns;
```

```
        float  spawn_interval;
```

```
        float  spawn_timer;
```

```
    public:
```

```
        monster_spawner(math::vec2 const p, float const z, engine::actor& target, geometry::map_data
```

```
        void update(float dt) override;
```

```
};
```

```
}
```

monster-maly-szybki.h

```
#pragma once
```

```
#include "entities/monster.h"
```

```
namespace entities {
```

```
    class monster_Maly_Szybki : public monster {
```

```
        bool is_dashing;
```

```
        float dash_timer;
```

```
        float dash_cooldown;
```

```
    public:
```

```
        monster_Maly_Szybki(math::vec2 const p, float const z, engine::actor& target, geometry::map_
```

```
        void update(float dt) override;
```

```
};
```

```
}
```

monster-elite-tank.h

```
#pragma once
```

```
#include "entities/monster.h"
```

```
namespace entities {
```

```
    class monster_elite_tank : public monster {
```

```
        bool    melee_mode;
```

```
        float   melee_threshold;
```

```
        float   heavy_timer;
```

```
        float   heavy_cd;
```

```
    public:
```

```
        monster_elite_tank(math::vec2 const p, float const z, engine::actor& target, geometry::map_d
```

```
        void update(float dt) override;
```

```
};
```

```
}
```

monster-ranged.h

```
#pragma once
```

```
#include "entities/monster.h"
```

```
namespace entities {
```

```
    class monster_ranged : public monster {
```

```
        float preferred_dist;
```

```
    public:
```

```
        monster_ranged(math::vec2 const p, float const z, engine::actor& target, geometry::map_data&
```

```
        void update(float dt) override;
```

```
    };
```

```
}
```

monster-all-rounder.h

```
#pragma once
```

```
#include "entities/monster.h"
```

```
namespace entities {
```

```
    class monster_all_rounder : public monster {  
        bool   melee_mode;  
        float  melee_threshold;
```

```
    public:
```

```
        monster_all_rounder(math::vec2 const p, float const z, engine::actor& target, geometry::map_
```

```
        void update(float dt) override;
```

```
    };
```

```
}
```

monster-assault.h

```
#pragma once
```

```
#include "entities/monster.h"
```

```
namespace entities {
```

```
    class monster_assault : public monster {
```

```
        int    burst_size;
```

```
        float  burst_interval;
```

```
        int    burst_remaining;
```

```
        float  burst_timer;
```

```
        float  burst_cooldown;
```

```
        float  strafe_sign;
```

```
        float  strafe_timer;
```

```
    public:
```

```
        monster_assault(math::vec2 const p, float const z, engine::actor& target, geometry::map_data
```

```
        void update(float dt) override;
```

```
};
```

```
}
```

monster-boss.h

```
#pragma once

#include "entities/monster.h"
#include "engine/world.h"
#include "geometry/map-data.h"
#include <vector>

namespace entities {

    class monster_boss : public monster {
        // phase tracking
        int current_phase = 1;
        float max_hp_val;

        // ranged: 4 hitscans with slow
        float ranged_cooldown = 0.0f;
        static constexpr float ranged_cd_max = 3.0f;
        static constexpr int projectile_count = 4;
        static constexpr float projectile_spread = 0.15f;

        // melee: burning
        // uses base attack_cooldown / attack_cd_max

        // charge (phase 2+3)
        bool is_charging = false;
        float charge_timer = 0.0f;
        float charge_cooldown = 0.0f;
        float post_charge_stun = 0.0f;
        math::vec2 charge_dir{0.0f, 0.0f};
        static constexpr float charge_speed_mult = 5.0f;
        static constexpr float charge_duration = 1.2f;
        static constexpr float charge_cd_max = 6.0f;
        static constexpr float stun_duration = 2.0f;

        // channel (phase 3)
        bool is_channeling = false;
        float channel_timer = 0.0f;
        float flash_timer = 0.0f;
        bool phase3_debuffed = false;

        // spawned minions tracking
        std::vector<util::indexed_storage<std::unique_ptr<engine::entity>>::id_t> minion_ids;

        // combat stance
        bool wants_melee = false;
        float stance_timer = 10.0f;

        geometry::map_data& boss_map_ref;

        float hp_ratio() const;
        void update_phase();
        void phase1_ai(float dt, float dist);
        void phase2_ai(float dt, float dist);
        void phase3_ai(float dt, float dist);
        void ranged_attack_slow();
    };
}
```



```

    void melee_attack_burn();
    void start_charge();
    void update_charge(float dt);
    void start_channel();
    bool minions_alive() const;

public:
    monster_boss(math::vec2 const p, float const z, engine::actor& target, geometry::map_data& m

    bool channeling() const;
    float flash_phase() const;

    void update(float dt) override;
};
}

```

monster-sniper.h

```
#pragma once
```

```
#include "entities/monster.h"
```

```
namespace entities {
```

```
    class monster_sniper : public monster {  
        float shoot_interval;  
        float aim_timer;
```

```
    public:
```

```
        monster_sniper(math::vec2 const p, float const z, engine::actor& target, geometry::map_data&  
        void update(float dt) override;
```

```
    };
```

```
}
```

plasma-projectile.h

```
#pragma once

#include "engine/projectile.h"

namespace entities {

    class plasma_projectile : public engine::projectile {
    public:
        using projectile::projectile;
    };

}
```

player.h

```
#pragma once

#include <vector>
#include <memory>

#include "engine/actor.h"
#include "math/vec2.h"
#include "combat/weapons/weapon.h"
#include "util/componentized.h"

namespace geometry { class map_data; }
namespace engine { class world; }

namespace entities {
    class player : public engine::actor, public util::componentized<player> {
    public:
        using util::componentized<player>::operator();
        using engine::actor::operator();

    protected:
        [[=util::component_field{}]] std::vector<std::unique_ptr<combat::weapons::weapon>> weapons;
        float sensitivity;
        [[=util::component_field{}]] int current_weapon_index;
        geometry::map_data* map_ref = nullptr;
        engine::world* world_ref = nullptr;
        float collision_radius = 16.0f;
        float walk_speed;
        float sprint_speed;

        friend class util::componentized<player>;

    public:

        player(math::vec2 const p, float const z, assets::texture_id const tex, float const is, float const fr);

        void equip(int slot, std::unique_ptr<combat::weapons::weapon> wpn);

        void update(float dt) override;
        void move(math::vec2 direction);
        void rotate(float yaw);
        void shoot();
        void reload();
        void switch_weapons(int index);
        void start_sprint();
        void stop_sprint();

        template<typename WeaponT>
        bool resupply(int amount) {
            for (auto& w : weapons) {
                if (!w) continue;
                if (dynamic_cast<WeaponT*>(&*w)) {
                    w->resupply(amount);
                    return true;
                }
            }
        }
    };
}
```

```
        return false;
    }
};
}
```

monster.h

```
#pragma once

#include "engine/actor.h"
#include "assets/ids.h"

namespace engine { class world; }
namespace geometry { class map_data; }

namespace entities {
    class monster : public engine::actor {
    protected:
        engine::actor& target;
        geometry::map_data& map_ref;
        engine::world& world_ref;
        float attack_cooldown;
        float attack_range;
        float detection_radius;
        float attack_damage;
        float attack_cd_max;
        float collision_radius = 16.0f;

        bool has_target() const;
        float dist_to_target() const;
        math::vec2 dir_to_target() const;
        void apply_wall_collision(math::vec2& new_pos) const;
        void move_toward_target(float speed, float dt);
        void move_away_from_target(float speed, float dt);
        void strafe(float speed, float dt);
        void melee_attack(float damage);
        void ranged_attack(float damage, assets::texture_id tex, float speed);

    public:

        monster(math::vec2 const p, float const z, assets::texture_id const tex, float const is, flo

        void update(float dt) override;
    };
}
```


6

7

8